

Programming Project 3 – Knapsack Optimization

Connor Callahan | CS 2430-501 | Summer 2026

The Knapsack Problem is an organizational approach problem the idea behind it is there are a weight value and a limit and the combination of candidate cannot go over the weight limit of 700KG of payload there are 12 ships/experiments the goal is to find the subset of ships that fit within the limits and gets to the highest weight as possible

This project was made in C++ and has 4 different approaches to complete the problem most of them are greedy approaches and try to make logically correct choices one chooses the highest rating then the other picks the lightest first and then the ratio composed of weight and rating and then the good old fashion brute force that will compare all of the possible candidates and then choose the best one this one will always get the most optimal result

Results/ output will be in a file named output.txt at "src/"

The greedy methods are faster and simpler for the computer or more efficient but they will miss some that the brute force will eventually find the most optimal pair but comes at a cost it will be fine with 12 as its only 4096 subsets but as soon as you start increasing that number say 30 you are already up to more than a billion subsets and this is just not practical taking a step further u can build sort of table with more likely possible solutions instead of comparing everything makes it much more manageable and lowers the complexity for the computer there are others which lean more towards the brute force but this displayed it is important to decide with methods is used as if you need most accurate you will one closer to the brute force as if you don't care as much and just need it fast you can choose one of the more greedy approaches

This program shows the differences between the greedy or fast approaches and the guaranteed optimal solutions for the absolute best case the more dynamic cases are the best as they are the middle ground of the 2 sides when you don't know how much work is going to be needed

The lesson i learned is there are 2 problems when it comes to comparison speed either computing power or time one or the other you can either expand into more cores of the cpu to split the work or you can wait the time it takes to compute this is the main problem when it comes to brute force and why it is necessary for the dynamic approaches by mixing the two

Sources: Course materials, CS 2430-501, Summer 2026, SLCC.